

Tugas Sistem Multimedia

Watermarking



Dipersiapkan oleh :

Anisa Aulia Alhaqi

18224080@std.stei.itb.ac.id

Program Studi Sistem dan Teknologi Informasi
Sekolah Teknik Elektro dan Informatika - Institut Teknologi Bandung
Jl. Ganesha 10, Bandung 40132



Program Studi
Sistem dan Teknologi Informasi
STEI – ITB

Daftar Isi |

Daftar Isi 	2
01 — Teori Singkat	3
└─ 1.1 Kompresi JPEG	3
└─ 1.2 Watermarking Digital	4
└─ 1.3 Metode LSB Klasik vs Robust LSB	4
02 — Perancangan dan Implementasi	5
└─ 2.1 Diagram Alur Kerja Sistem	5
└─ 2.2 Kronologi Pengembangan	6
└─ 2.2.1 Eksperimen Full Color LSB	6
└─ 2.2.2 Eksperimen Grayscale LSB	6
└─ 2.1.3 Eksperimen Color Host dan Binary Watermark	6
└─ 2.3 Implementasi Manual JPEG	7
└─ 2.4 Optimasi Robustness	7
03 — Pengujian dan Analisis Hasil Program	8
└─ 3.1 Deskripsi Input dan Output Visual	8
└─ 3.2 Tabel Hasil Pengujian	9
└─ 3.3 Perbandingan Antar Eksperimen	10
└─ 3.4 Analisis Ketahanan	10
04 — Lampiran	11
└─ 4.1 Pranala Repository	11

01 — Teori Singkat

└ 1.1 Kompresi JPEG

JPEG (Joint Photographic Experts Group) adalah standar kompresi citra lossy yang sangat umum digunakan. Kompresi ini bekerja dengan mengeksploitasi keterbatasan persepsi mata manusia, membuang informasi detail frekuensi tinggi dan melakukan subsampling pada informasi krominan (warna), sementara mempertahankan informasi kecerahan.

Secara teknis, proses kompresi JPEG berjalan sebagai berikut:

1. Citra dibagi menjadi blok-blok kecil berukuran 8×8 piksel.
2. Setiap blok ditransformasi menggunakan Discrete Cosine Transform (DCT), mengubah data spasial menjadi representasi domain frekuensi.
3. Koefisien frekuensi hasil DCT dibagi dengan nilai pada Standard JPEG Quantization Table yang telah diskalakan oleh Quality Factor (QF). Pembagian ini membulatkan banyak koefisien frekuensi tinggi menjadi nol.
4. Hasil kuantisasi di-decode kembali melalui Inverse DCT (IDCT) untuk mendapatkan piksel spasial yang telah mengalami perubahan.



Gambar 1.1 Alur kerja kompresi JPEG

Semakin kecil nilai QF, semakin agresif proses kuantisasi membuang data, sehingga menghasilkan ukuran file yang lebih kecil dengan degradasi kualitas yang lebih besar.

└ 1.2 Watermarking Digital

Digital watermarking adalah teknik menyisipkan informasi tersembunyi ke dalam sebuah media digital (citra, audio, video) untuk tujuan pembuktian kepemilikan, pelacakan distribusi, atau autentikasi. Watermark yang ideal harus memenuhi dua kriteria utama:

1. Imperceptibility. Artinya, keberadaan watermark tidak merusak kualitas visual citra host dan tidak dapat dideteksi secara visual oleh mata manusia.
2. Robustness, yaitu watermark harus mampu bertahan dari berbagai manipulasi atau serangan pemrosesan sinyal, seperti kompresi JPEG, pemfilteran, atau penskalaan.

└ 1.3 Metode LSB Klasik vs Robust LSB

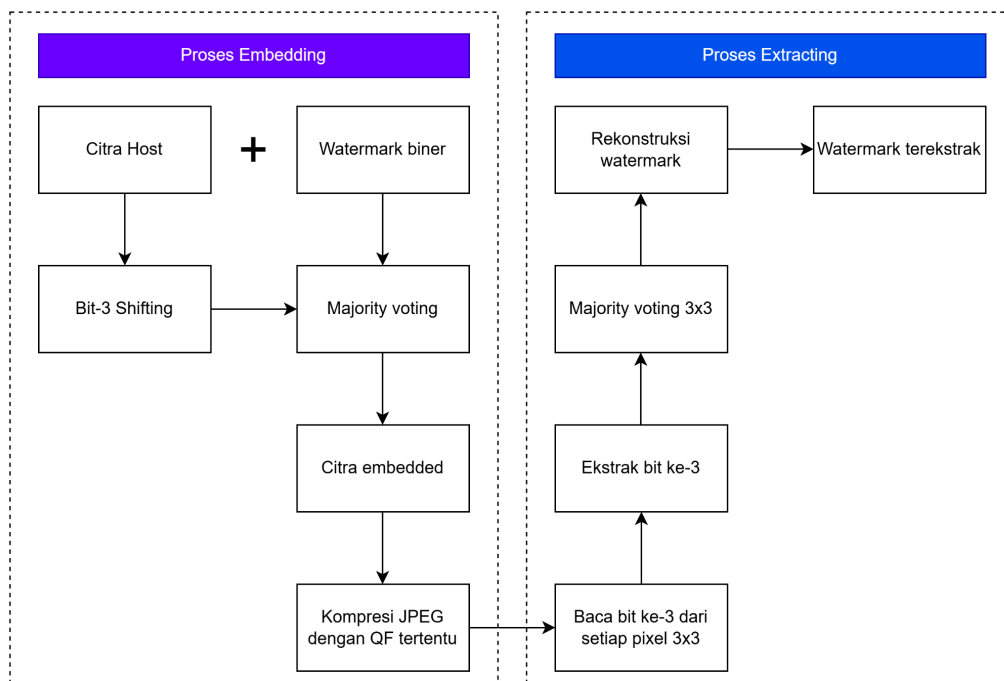
Metode Least Significant Bit (LSB) klasik bekerja dengan mengganti bit paling kanan (bit ke-0) dari sebuah nilai piksel dengan bit data watermark. Keunggulannya imperceptibility yang sangat tinggi. Namun, ia sangat rapuh. Kompresi lossy seperti JPEG secara otomatis membuang detail halus pada bit-bit rendah, sehingga watermark LSB klasik akan hancur saat citra dikompresi.

Untuk mengatasinya, diperlukan teknik Robust LSB. Pendekatan ini memindahkan penyisipan ke *bit-plane* yang lebih tinggi (seperti bit ke-3 atau ke-4) sehingga perubahan nilainya cukup signifikan untuk bertahan dari pembulatan kuantisasi. Selain itu, redundansi spasial dimanfaatkan dengan menyebarkan satu bit ke beberapa piksel sekaligus, sehingga meskipun sebagian piksel terdistorsi, nilai bit asli masih bisa dipulihkan melalui voting mayoritas.

02 — Perancangan dan Implementasi

Sistem yang dibangun bertujuan untuk menyisipkan citra biner (logo) ke dalam citra berwarna (wajah) sedemikian rupa, sehingga citra host tetap berwarna secara visual, tetapi watermark di dalamnya mampu diekstrak kembali meskipun citra tersebut telah dikompresi menjadi JPEG dengan QF yang sangat rendah.

— 2.1 Diagram Alur Kerja Sistem



Gambar 2.1 Alur kerja keseluruhan sistem

— 2.2 Kronologi Pengembangan

Metode final ditemukan melalui tiga tahap eksperimen yang masing-masing mengungkap kelemahan berbeda, meliputi:

└ 2.2.1 Eksperimen Full Color LSB

Pada percobaan pertama, watermark berwarna disisipkan ke dalam foto *host* berwarna menggunakan LSB Bit-0 standar pada ketiga kanal warna (R, G, B). Hasilnya, watermark langsung rusak total dan berubah menjadi derau visual (*noise*) segera setelah citra disimpan ke format JPEG, bahkan pada QF 100 sekalipun. Ini secara langsung membuktikan bahwa LSB Bit-0 sama sekali tidak kompatibel dengan kompresi lossy karena kuantisasi DCT membuang persis bit-bit rendah tempat watermark disisipkan.

└ 2.2.2 Eksperimen Grayscale LSB

Menyadari bahwa masalahnya ada pada sensitivitas kanal warna, percobaan kedua dilakukan dengan mengubah baik foto *host* maupun watermark menjadi *grayscale* (hitam-putih) sebelum proses embedding. Hasilnya, watermark berhasil diekstrak dengan cukup jelas. Namun, solusi ini memiliki kelemahan praktis yang signifikan: foto *host* kehilangan seluruh informasi warnanya. Untuk kebutuhan nyata di sistem multimedia, menyajikan foto wajah dalam format hitam-putih tidak ideal dan tidak memenuhi ekspektasi pengguna modern.

└ 2.1.3 Eksperimen Color Host dan Binary Watermark

Solusi terbaik ditemukan dengan menggabungkan keunggulan dari dua eksperimen sebelumnya: foto *host* dibiarkan tetap berwarna (RGB) untuk mempertahankan kualitas visual, sementara watermark dikonversi menjadi format biner (hitam-putih). Dengan format biner, setiap bit watermark hanya memiliki dua kemungkinan nilai (0 atau 1), sehingga teknik *majority voting* berbasis blok dapat diterapkan dengan sangat efektif. Kombinasi ini menghasilkan keseimbangan optimal antara kualitas visual foto dan ketahanan watermark.

└ 2.3 Implementasi Manual JPEG

1. Citra dibagi menjadi blok kecil berukuran 8x8
2. Setiap blok dihitung nilai DCT-nya menggunakan formula:

$$F(u, v) = \frac{1}{4} C(u)C(v) \sum_{x=0}^7 \sum_{y=0}^7 f(x, y) \cos \left[\frac{(2x+1)u\pi}{16} \right] \cos \left[\frac{(2y+1)v\pi}{16} \right]$$

3. Setiap koefisien frekuensi dibagi dengan matriks kuantisasi standar JPEG (Q-Table) yang telah dikalikan dengan Quality Factor (QF) tertentu
4. Dilakukan de-kuantisasi dan Inverse DCT (IDCT) untuk mendapatkan kembali piksel yang telah mengalami perubahan

└ 2.4 Optimasi Robustness

Dilakukan beberapa cara untuk menjamin data setelah proses sebelumnya, meliputi:

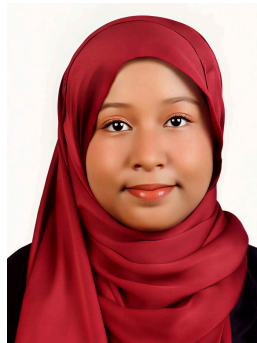
1. Target Kanal Hijau, meliputi penyisipan dilakukan khusus pada kanal Hijau karena bobot kontribusinya paling besar terhadap nilai Luminans (Y), yaitu komponen yang paling dijaga integritasnya oleh algoritma kompresi JPEG.
2. Bit-Plane Shifting (Bit-3), yaitu penyisipan dilakukan pada bit ke-3 (bukan bit ke-0) agar nilainya cukup signifikan untuk tidak hilang saat dibulatkan oleh kuantisasi DCT
3. 3x3 Majority Voting, yaitu satu bit watermark disebar ke dalam 9 piksel (blok 3x3). Saat ekstraksi, diambil suara terbanyak dari 9 piksel tersebut untuk menentukan nilai bit yang asli

03 — Pengujian dan Analisis Hasil Program

— 3.1 Deskripsi Input dan Output Visual

Input:

- **Citra Host:** [foto_wajah.jpg](#), foto wajah berwarna (RGB). Dipilih karena merepresentasikan kasus penggunaan nyata yang paling umum.



Gambar 3.1 Input Citra Host

- **Watermark:** [input.png](#), logo dalam format biner yang digunakan sebagai data yang disisipkan.



Gambar 3.2 Input Watermark

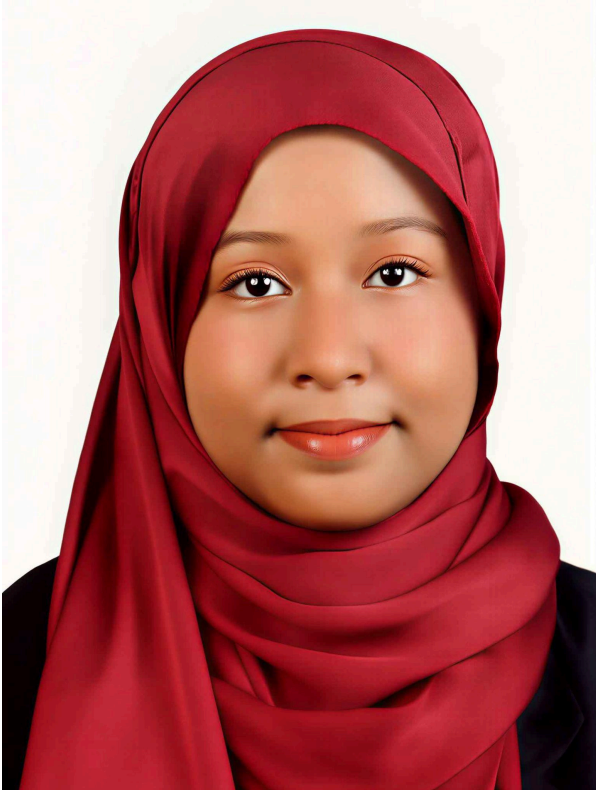
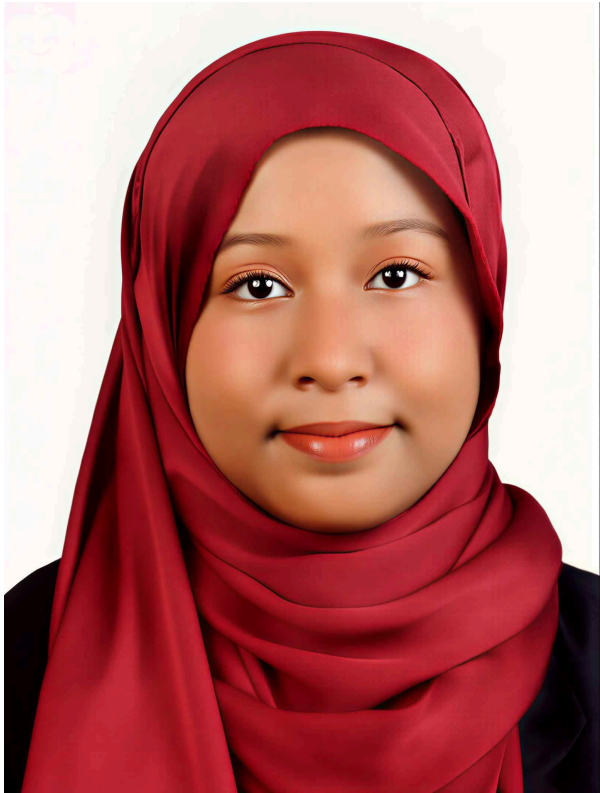
Output:

- **Citra Ter-embed:** Secara visual identik dengan foto asli. Tidak ada artefak atau derau yang terlihat mata, dikonfirmasi dengan nilai PSNR di atas 53 dB pada QF 100.

- **Watermark Terekstrak per QF:** Serangkaian gambar hasil ekstraksi pada berbagai nilai QF (100, 90, 70, 50, 30, 10) yang memperlihatkan gradasi ketahanan watermark secara visual.

— 3.2 Tabel Hasil Pengujian

Tabel 3.1 Tabel Hasil Embedding Watermark

Sebelum	Setelah
	

Tabel 3.2 Tabel Ekstraksi Watermark

QF JPEG	PSNR Img (dB)	BER	Bit Error	Hasil
---------	---------------	-----	-----------	-------

100	53.60	0.0000	0	
90	51.11	0.0005	2	
80	49.8	0.0015	6	
70	49.36	0.0024	9	
50	47.52	0.0076	31	
30	44.55	0.0334	136	
10	36.17	0.2681	1098	

— 3.3 Perbandingan Antar Eksperimen

Implementasi manual DCT membuktikan bahwa proses kuantisasi pada Quality Factor rendah (seperti QF 30) memberikan dampak kerusakan yang nyata pada bit-bit rendah piksel. Namun, dengan penggunaan Majority Voting 3x3 dan Penyisipan pada Bit-3, Bit Error Rate (BER) dapat ditekan hingga di bawah 1% pada QF 50, dan hanya sekitar 3.3% pada QF 30. Hal ini

membuktikan bahwa algoritma Robust LSB yang dirancang sangat efektif bertahan terhadap algoritma kompresi JPEG yang diimplementasikan secara matematis.

└ 3.4 Analisis Ketahanan

Hasil pengujian memperlihatkan pola yang konsisten dan dapat dijelaskan secara matematis. Pada rentang QF 100 hingga QF 50, sistem bekerja dengan sangat baik. BER tidak pernah melebihi 0,76%, artinya dari ribuan bit watermark, kurang dari 1% yang salah setelah kompresi. Pada level ini, kuantisasi DCT belum cukup agresif untuk menggeser nilai bit ke-3 secara permanen, dan *majority voting* 3×3 dengan mudah mengoreksi sisa kesalahan kecil yang terjadi.

Pada QF 30, terjadi lonjakan BER ke 3,34% (136 bit error). Meski angka ini meningkat signifikan, watermark masih terbaca jelas secara visual. Ini adalah bukti langsung efektivitas *majority voting*: meskipun kuantisasi yang sangat agresif berhasil membalik beberapa bit dalam blok 3×3, mayoritas dari 9 piksel dalam setiap blok masih membawa nilai yang benar.

Pada QF 10, sistem mulai kewalahan dengan BER 26,81% (1098 bit error). Pada tingkat kompresi ekstrem ini, kuantisasi DCT membuang hampir seluruh detail frekuensi, bahkan bit ke-3 pun tidak cukup signifikan untuk bertahan. *Majority voting* tidak lagi mampu mengkompensasi kerusakan yang terjadi hampir merata di semua piksel blok. Hasil visual watermark yang terekstrak masih menampilkan pola samar dari logo asli, namun sudah terdistorsi cukup parah.

Secara keseluruhan, sistem terbukti efektif untuk penggunaan praktis pada $QF \geq 30$, yang mencakup sebagian besar skenario kompresi JPEG di dunia nyata.

04 — Lampiran

└ 4.1 *Pranala Repository*

<https://github.com/anisaalhaqi/Kripto>